



PROBAR

PROBAR ANTES DE QUE SEA PARA SIEMPRE

Hardhat, pruebas automatizadas y despliegue reproducible: el flujo real de un equipo que no puede publicar parches.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Hoy se sale del navegador. Y se aprende la diferencia entre tener pruebas y tener pruebas que atrapan errores. Pausa de 10 minutos entre A y B.

A**EL FLUJO PROFESIONAL**

Proyecto, red local, console.log, pruebas, cobertura, mutación, despliegue y secretos.

~65 min**B****LABORATORIO 08 · CAZAR MUTANTES**

Suite de ≥ 10 pruebas, ≥ 80 % de cobertura, tres mutantes muertos, Ignition en Sepolia.

~80 min**C****EL PROYECTO, EN HARDHAT**

Estructura del repositorio del equipo, .gitignore y lo que nunca se sube.

~25 min



AL SALIR DE AQUÍ

Trabajar con el flujo real de un equipo: pruebas, cobertura, mutación, despliegue reproducible y código verificado.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Qué no se puede hacer bien desde Remix?
- 02** ¿Qué hace que una prueba sea buena, y no solo verde?
- 03** ¿Por qué el 80 % de cobertura puede dejar pasar un error grave?
- 04** ¿Cómo se despliega sin que la clave privada toque el repositorio?

RA4 · Construir una suite de pruebas automatizadas y desplegar contratos verificados con un entorno profesional (Hardhat).



SIGLAS Y TÉRMINOS DE HOY

HARDHAT*entorno de desarrollo*

Compila, prueba, despliega y verifica desde la terminal. Trae su propia red local.

FIXTURE*estado de partida*

Despliegue que se ejecuta una vez y se restaura como instantánea antes de cada prueba.

MUTANTE*mutation testing*

Copia del contrato con un error introducido a propósito. Una buena suite debe fallar contra él.

RPC*Remote Procedure Call*

La URL del nodo con el que se habla. Suele llevar una clave de API: es un secreto.

MOCHA · CHAI*bibliotecas de pruebas*

Mocha organiza (describe, it); Chai comprueba (expect). Estándar en JavaScript.

COBERTURA*code coverage*

Porcentaje de líneas del contrato que alguna prueba ejecutó. No dice si se comprobaron.

IGNITION*Hardhat Ignition*

Despliegue declarativo: registra lo hecho y retoma si se interrumpe.

KEYSTORE*almacén cifrado*

Donde Hardhat 3 guarda claves y URLs, protegido con contraseña, fuera del repositorio.

A



APROXIMADAMENTE 65 MINUTOS

EL FLUJO PROFESIONAL

En software tradicional una prueba ahorra tiempo. Aquí es la última oportunidad de encontrar un error antes de que custodie dinero para siempre.

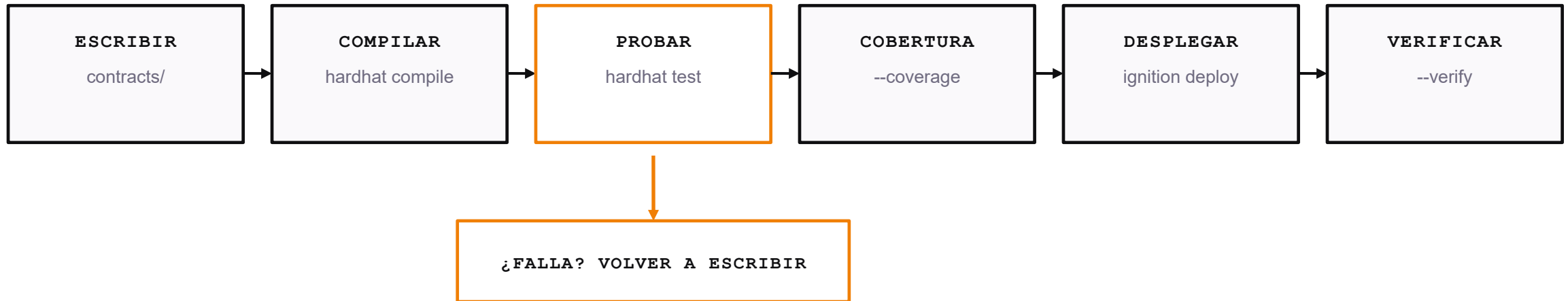


LO QUE REMIX NO PUEDE HACER BIEN

NECESIDAD	EN REMIX	EN HARDHAT
Repetir 50 pruebas tras cada cambio	A mano, clic por clic.	Un comando, en segundos.
Probar qué pasa dentro de una semana	Imposible: el tiempo es real.	Se adelanta el reloj de la red local.
Saber qué líneas nunca se ejecutaron	No existe.	Informe de cobertura.
Trabajar en equipo con Git	Archivos en el navegador.	Repositorio normal, revisión de código.
Desplegar igual en local, prueba y producción	Cada vez a mano.	Módulo de despliegue con parámetros.
Guardar claves con seguridad	Depende de la billetera del navegador.	Almacén cifrado fuera del proyecto.

Remix sigue siendo excelente para explorar y enseñar. Pero ningún equipo serio despliega desde el navegador.

DEL CÓDIGO AL CONTRATO VERIFICADO



TODO EN LA RED LOCAL

Escribir, compilar, probar y medir cobertura no gasta ETH ni necesita internet. Se repite cien veces al día.

SOLO AL FINAL, LA RED PÚBLICA

Desplegar y verificar se hace una vez, cuando la suite está verde y la cobertura es suficiente.



ESTRUCTURA Y CONFIGURACIÓN

```
laboratorios-evm/  
├── contracts/           contratos .sol  
├── test/                pruebas .test.js  
├── ignition/  
│   ├── modules/        cómo se despliega  
│   └── parametros/      con qué valores, por red  
├── scripts/             utilidades  
├── hardhat.config.js  
├── package.json          versiones EXACTAS  
└── .gitignore            lo que NUNCA se sube
```

HARDHAT.CONFIG.JS (EXTRACTO)

```
export default {  
  plugins: [toolbox],  
  solidity: {  
    version: "0.8.34",  
    settings: { optimizer: { enabled: true, runs: 200 } },  
  },  
  networks: {  
    sepolia: {  
      type: "http",  
      url: configVariable("SEPOLIA_RPC_URL"),  
      accounts: [configVariable("SEPOLIA_PRIVATE_KEY")],  
    },  
  },  
};
```

configVariable no contiene el secreto: le dice a Hardhat que lo busque en el almacén cifrado solo cuando se use esa red. Las pruebas locales nunca piden.

UNA CADENA COMPLETA EN SU COMPUTADOR

CARACTERÍSTICA	QUÉ PERMITE
20 cuentas con 10 000 ETH de prueba	Simular a Ana, Beto, el dueño y un atacante sin pedir fondos a nadie.
Minado instantáneo	Cada transacción queda en un bloque al momento: las pruebas corren en milisegundos.
Control del reloj	time.increase(7 días) prueba plazos sin esperar una semana.
Instantáneas	Guardar el estado y volver a él: la base de los fixtures.
Mensajes de error completos	Muestra qué error personalizado revirtió y con qué argumentos.
<code>npx hardhat node</code>	La misma red, como servidor en localhost:8545, para conectar una billetera o una interfaz (S12).


 LAS CUENTAS DE LA RED LOCAL SON PÚBLICAS

Sus claves privadas están impresas en la documentación. Nunca se usan en una red real.



CONSOLE.LOG DENTRO DE SOLIDITY

```
import "hardhat/console.sol";

function aportar() external payable {
    console.log("aporte de %s por %d wei", msg.sender, msg.value);
    total += msg.value;
    console.log("total ahora: %d", total);
}
```

SALIDA REAL DE NPX HARDHAT TEST

```
aporte de 0x70997970c51812dc3a010c7d01b50e0d17dc79c8 por 1000 wei
total ahora: 1000
✓ imprime
```

- Solo funciona en la red local de Hardhat: en Sepolia no imprime nada.
- Sirve para ver valores intermedios mientras se escribe una prueba que no entienden por qué falla.
- Cuesta gas y ensucia el contrato: se borra antes de desplegar.

Error típico: dejar el import y los console.log en el contrato que se verifica en el explorador.



VERSIONES EXACTAS, O SORPRESAS A MITAD DE SEMESTRE

PACKAGE.JSON

```
npm install --save-exact @openzeppelin/contracts@5.6.1

"devDependencies": {
  "hardhat": "3.16.0",
  "@nomicfoundation/hardhat-toolbox-mocha-ethers": "3.0.7"
},
"dependencies": { "@openzeppelin/contracts": "5.6.1" }
```

ESCRITURA	INSTALA
"5.6.1"	Exactamente esa.
"^5.6.1"	Cualquier 5.x posterior.
"~5.6.1"	Cualquier 5.6.x posterior.

Sin --save-exact, npm escribe con ^: dos personas del mismo equipo pueden compilar con bibliotecas distintas.



POR QUÉ IMPORTA MÁS AQUÍ QUE EN UNA APLICACIÓN WEB

El código de OpenZeppelin que se compila queda dentro del contrato desplegado, para siempre. Si una versión nueva cambia un comportamiento, no hay actualización posible. Por eso package-lock.json también se sube al repositorio: fija incluso las dependencias de las dependencias.

DESCRIBE, IT, EXPECT

```
import { expect } from "chai";
import { network } from "hardhat";

const { ethers, networkHelpers } = await network.create();

async function desplegar() {                                // fixture: con NOMBRE
  const [dueno, ana] = await ethers.getSigners();
  const r = await ethers.deployContract("Recaudo", [META, PLAZO]);
  return { r, dueno, ana };
}

describe("Recaudo · aportar", () => {                       // grupo
  it("rechaza un aporte de cero", async () => {              // caso
    const { r, ana } = await networkHelpers.loadFixture(desplegar);
    await expect(r.connect(ana).aportar({ value: 0 }))
      .to.be.revertedWithCustomError(r, "AporteCero");      // comprobación
  });
});
```

CADA PRUEBA EMPIEZA DE CERO, SIN PAGARLO

- 01 PRIMERA LLAMADA** `loadFixture` ejecuta `desplegar()`: despliega el contrato y guarda una instantánea de toda la cadena.
- 02 LLAMADAS SIGUIENTES** No vuelve a desplegar: restaura la instantánea. Milisegundos en lugar de segundos.
- 03 AISLAMIENTO** Lo que una prueba cambió —aportes, tiempo adelantado— no existe para la siguiente.

EL ERROR CLÁSICO

Desplegar en un `before()` compartido y dejar que las pruebas se afecten entre sí. La suite pasa en un orden y falla en otro, y nadie entiende por qué. Con fixtures ese problema no puede ocurrir.



LAS CINCO FAMILIAS DE PRUEBAS

FAMILIA	PREGUNTA QUE RESPONDE	EJEMPLO CON LA SUBASTA DE LA S7
Caso feliz	¿Hace lo que promete cuando todo va bien?	Una puja válida cambia mejorPostor y emite NuevaPuja.
Reversión	¿Se niega a lo que no debe hacer?	Pujar por debajo del mínimo revierte con PujalInsuficiente.
Permisos	¿Solo quien debe puede hacerlo?	Solo el dueño pausa.
Eventos	¿Le cuenta al mundo lo que pasó, con los datos correctos?	SubastaFinalizada con ganador y monto.
Bordes	¿Qué pasa exactamente en el límite?	La puja que supera en exactamente el 1 %, ni un wei menos.

Las cuatro primeras dan cobertura. La quinta es la que atrapa los errores.



MATCHERS DE HARDHAT PARA CHAI

```
// revierte con un error personalizado, y con qué argumentos
await expect(tx).to.be.revertedWithCustomError(s, "PujaInsuficiente").withArgs(minimo, enviado);

// emite un evento con argumentos exactos
await expect(tx).to.emit(r, "Aporte").withArgs(ana.address, monto, total);

// cambia el saldo de ETH de una o varias cuentas (ethers va primero)
await expect(tx).to.changeEtherBalance(ethers, ana, monto);
await expect(tx).to.changeEtherBalances(ethers, [ana, r], [monto, -monto]);

// sin permiso, con el error de OpenZeppelin
await expect(tx).to.be.revertedWithCustomError(r, "OwnableUnauthorizedAccount");
```

DOS TRAMPAS

changeEtherBalance ya descuenta el gas: comparen el monto exacto. Y no usen .to.be.reverted a secas: está desaconsejado y no dice POR QUÉ revirtió.



OTRAS CUENTAS Y OTRO MOMENTO

```
const [dueno, ana, atacante] = await ethers.getSigners();

await r.connect(ana).aportar({ value: eth(3) });           // Ana firma

await time.increase(7 * 24 * 60 * 60);                    // pasa una semana
await time.increaseTo(fin);                                // mina un bloque en fin
await time.setNextBlockTimestamp(fin);                     // el PRÓXIMO bloque cae en fin

await expect(r.connect(atacante).reclamarFondos())         // alguien sin permiso
  .to.be.revertedWithCustomError(r, "OwnableUnauthorizedAccount");
```

connect(cuenta) cambia quién firma. time viene de networkHelpers y mueve el reloj de la red local, nunca el de una red real. La diferencia entre increaseTo y setNextBlockTimestamp es sutil y es la lámina siguiente.



¿EN QUÉ SEGUNDO CAE MI TRANSACCIÓN?

La Subasta de la S7 dice: `if (block.timestamp >= fin) revert SubastaCerrada()`. ¿Qué pasa si alguien puja en el segundo exacto de fin? Hay que poder llevar una transacción a ese segundo.

HERRAMIENTA	QUÉ HACE EL RELOJ	LA PUJA SIGUIENTE CAE EN	SIRVE PARA PROBAR
<code>time.increaseTo(fin)</code>	Mina un bloque vacío con marca fin.	<code>fin + 1</code>	«después del cierre»
<code>time.setNextBlockTimestamp(fin)</code>	No mina nada: fija la marca del PRÓXIMO bloque.	<code>fin</code> , exacto	«en el segundo exacto»
<code>setNextBlockTimestamp(fin - 1n)</code>	Igual, un segundo antes.	<code>fin - 1</code>	«un segundo antes»

[] LA REGLA PARA CUALQUIER PLAZO

Para cada comparación con el tiempo, tres pruebas: un segundo antes, el segundo exacto y un segundo después. Una prueba con «una semana después» nunca llega al límite: por eso casi nadie lo prueba, y por eso ahí viven los errores.



QUÉ MIDE, Y QUÉ NO MIDE

```
npx hardhat test test/s07/Subasta.test.js --coverage
```

QUÉ MIDE

- Qué líneas del contrato ejecutó alguna prueba.
- Qué líneas nunca se ejecutaron: el informe las lista por número.
- Útil para encontrar funciones o reversiones que nadie probó.

QUÉ NO MIDE

- Si la prueba comprobó el resultado. Una prueba sin expect da cobertura.
- Si se probó el valor exacto del borde.
- Si el contrato hace lo que el negocio necesita.

La cobertura dice dónde NO hay pruebas. No dice que las que hay sirvan.

UNA FILA, TRES COLUMNAS, UNA LISTA

SALIDA REAL, RECORTADA

File Path	Line %	Statement %	Uncovered Lines
contracts\s07\Empaquetado.sol	100.00	100.00	-
contracts\s07\Subasta.sol	97.06	97.87	118
contracts\s08\Recaudo.sol	0.00	0.00	49-51, 55-56, ...
Total	21.38	21.64	

- 01

BUSCAR SU FILA

El informe lista TODOS los contratos del repositorio. La fila que importa es la del contrato que prueban.
- 02

IGNORAR EL TOTAL

21 % de Total no significa nada: incluye contratos de otras sesiones que esta suite ni toca.
- 03

LEER LAS LÍNEAS

118 es el revert EnvioFallido de retirar(): solo se ejecuta si el receptor rechaza el pago. Cada número es una pregunta.

También queda un informe navegable en `coverage/html/index.html`, con cada línea pintada de verde o rojo.

¿MIS PRUEBAS NOTARÍAN UN ERROR?

La idea: introducir un error pequeño a propósito —un mutante— y correr la suite. Si alguna prueba falla, el mutante está MUERTO: la suite lo vio. Si todas pasan, está VIVO: ese error llegaría a producción.

MUTANTE SOBRE LA SUBASTA DE LA S7	RESULTADO CON LAS 23 PRUEBAS	POR QUÉ
<code>INCREMENTO_MINIMO_BPS = 10</code> en lugar de 100	MUERTO · 2 pruebas fallan	Dos pruebas comprueban el valor exacto: 2,02 ETH.
<code>Borrar emit Retiro(...)</code> en <code>retirar()</code>	VIVO · 23 pasan	Ninguna prueba comprueba el evento Retiro.

LO QUE ENSEÑA EL MUTANTE VIVO

Subasta.sol tiene 97 % de cobertura y aun así una interfaz que escuche el evento Retiro dejaría de enterarse de los retiros, sin que ninguna prueba lo note. La línea del evento estaba cubierta; nadie comprobaba lo que emitía. Arreglarlo es una línea: `to.emit(s, "Retiro").withArgs(ana.address, eth(1))`.

88 % DE COBERTURA Y TRES ERRORES VIVOS

Preparando el laboratorio escribimos una suite de referencia para el contrato Recaudo y tres copias del contrato con un error cada una. Después quitamos solo las pruebas de borde.

13
pruebas en verde contra el contrato correcto

88 %
de cobertura de líneas: cumple la meta del curso

13 / 13
siguen en verde contra CADA mutante

3
errores reales que la suite no ve

	SUITE SIN BORDES	SUITE CON 3 PRUEBAS DE BORDE MÁS
Contra el contrato correcto	13 pasan · 88 %	16 pasan · 92 %
Contra cada mutante	13 pasan · VIVO	15 pasan, 1 falla · MUERTO

Medido con `npx hardhat test --coverage y scripts/s08/cazar-mutantes.js`. Cuáles son los bordes: eso lo descubren ustedes.



DÓNDE VIVEN LOS ERRORES

BORDE	LA PREGUNTA QUE HAY QUE HACERLE AL CONTRATO	POR QUÉ AHÍ
Tiempo	¿Qué pasa un segundo antes, en el segundo exacto y un segundo después de cada plazo?	Una comparación mal escrita deja entrar o salir justo en el límite, y ninguna prueba «normal» llega a ese instante.
Umbral	¿Qué pasa con el valor exacto de cada límite: ni un wei más, ni uno menos?	Los casos cómodos siempre quedan lejos del límite: por eso nadie los prueba.
Repetición	¿Qué pasa si la misma cuenta llama dos veces seguidas?	Un efecto que debería ocurrir una vez puede repetirse si el estado no cambió.
Cero y vacío	¿Qué pasa con 0, con la dirección cero, con un texto vacío?	Son valores válidos para la EVM y absurdos para el negocio.
Quién llama	¿Qué pasa si llama otra cuenta, o un contrato?	Basta una función sin su modificador.

Para cada función del contrato, recorrer las cinco filas. Así se encuentran los errores que la cobertura no ve.



CUÁNTO CUESTA CADA FUNCIÓN, DE UN VISTAZO

```
npx hardhat test test/s07/Subasta.test.js --gas-stats
```

```
contracts/s07/Subasta.sol:Subasta
```

Function name	Min	Average	Median	Max	#calls
finalizar	49379	67786	73922	73922	4
pujar	45421	115482	124090	138289	24
retirar	30000	30000	30000	30000	5
estado	23494	23509	23516	23516	3
Deployment	712669				1

- pujar varía de 45 421 a 138 289: estrenar ranuras (primer postor, nuevo participante) es lo caro (S5 y S7).
- estado aparece con gas porque la prueba la llama; desde una aplicación, leer es gratis.
- Si un cambio de código hace subir mucho una cifra, es una señal para revisar antes de desplegar.



IGNITION: DESPLIEGUES QUE SE PUEDEN REPETIR

IGNITION/MODULES/RECAUDO.JS

```
import { buildModule } from "@nomicfoundation/hardhat-ignition/modules";

export default buildModule("RecaudoModulo", (m) => {
  const meta = m.getParameter("meta", 10n * 10n ** 18n);
  const duracion = m.getParameter("duracionSegundos", 604800);
  const recaudo = m.contract("Recaudo", [meta, duracion]);
  return { recaudo };
});
```

PARAMETROS/RECAUDO-SEPOLIA.JSON

```
{
  "RecaudoModulo": {
    "meta": "5000000000000000",
    "duracionSegundos": 1800
  }
}
```

- Describe QUÉ se despliega, no los pasos: Ignition decide el orden.
- Anota cada despliegue en ignition/deployments/: si se cae la red a mitad, retoma sin desplegar dos veces.
- El mismo módulo sirve para la red local y para Sepolia; cambian solo los parámetros (0,005 ETH y 30 minutos en Sepolia).



LA CLAVE PRIVADA NUNCA TOCA EL REPOSITORIO

```
npx hardhat keystore set SEPOLIA_RPC_URL      # la primera vez pide crear una contraseña
npx hardhat keystore set SEPOLIA_PRIVATE_KEY  # la de la billetera DEL CURSO
npx hardhat keystore set ETHERSCAN_API_KEY
npx hardhat keystore list                     # muestra los NOMBRES, nunca los valores
```

QUÉ HACE EL ALMACÉN

- Guarda los valores cifrados con una contraseña que se pide al usarlos.
- Vive en un archivo de su carpeta de usuario, fuera del proyecto.
- configVariable los busca solo cuando una tarea los necesita.

SI UNA CLAVE SE SUBE A GITHUB

- Hay programas que recorren los repositorios públicos buscando claves.
- ConsenSys documenta billeteras vaciadas «en segundos».
- Borrar el archivo no basta: queda en el historial de Git.

Si ocurre: la clave se da por perdida. Se mueven los fondos a una billetera nueva y no se vuelve a usar.

PUBLICAR EL CÓDIGO FUENTE DEL CONTRATO

```
npx hardhat ignition deploy ignition/modules/Recaudo.js \  
  --network sepolia \  
  --parameters ignition/parametros/recaudo-sepolia.json \  
  --verify
```

01 QUÉ HACE

Envía al explorador el código, la versión del compilador y la configuración. El explorador recompila y compara con el bytecode desplegado.

02 SI COINCIDE

Aparece la marca de verificación, el código legible y las pestañas para leer y escribir el contrato.

03 POR QUÉ IMPORTA

Un contrato sin verificar pide confianza ciega: nadie puede saber qué hace sin descompilarlo.

En la rúbrica del proyecto: sin contrato verificado, «pruebas y despliegue» cae a insuficiente.



ANTES DE LA PAUSA: CUATRO PREGUNTAS

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 Una suite tiene 95 % de cobertura. Den un ejemplo concreto de error que podría no detectar.

02 ¿Qué diferencia hay entre `time.increaseTo(fin)` y `time.setNextBlockTimestamp(fin)` para la transacción siguiente?

03 ¿Qué comando usan para ver qué líneas de su contrato nunca se ejecutaron, y qué fila del informe miran?

04 ¿Dónde vive la clave privada de Sepolia en este flujo, y qué archivo del repositorio la menciona?

B



APROXIMADAMENTE 80 MINUTOS · EN PAREJAS

LABORATORIO 08 - CAZAR MUTANTES

Escribir la suite de un contrato que no trae pruebas, alcanzar la cobertura, matar tres mutantes y desplegar con Ignition.



EL ENTORNO DE LA SALA, COMPROBADO

```
node --version           # v22.x
npm install              # solo si no se hizo en la S5
npx hardhat --version    # 3.16.0
npx hardhat test test/s07/Subasta.test.js # 23 passing: el entorno funciona
```

SI PASA ESTO	HACER ESTO
<code>npm install</code> tarda o falla por la red de la sala	Reintentar una vez; si persiste, avisar: el docente tiene el repositorio con node_modules en memoria USB.
<code>npx hardhat --version</code> no da 3.16.0	No actualizar nada. Revisar que la terminal esté en laboratorios-evm.
La suite de la S7 falla	Dejaron su versión incompleta de Subasta.sol: git checkout contracts/s07/Subasta.sol.

Cómo saber que terminó: 23 passing en la S7. Nada de lo que sigue funciona si esto no funciona.



LEER RECAUDO, COPIAR LA PLANTILLA

```
mkdir test/s08 # cmd: mkdir test\s08
cp andamiaje/s08/Recaudo.test.js test/s08/Recaudo.test.js
npx hardhat test test/s08/Recaudo.test.js # 2 passing: los ejemplos
```

FINAL	CONDICIÓN, TRAS EL PLAZO	QUÉ SE PUEDE HACER
Exitoso	totalRecaudado >= meta	El dueño reclama los fondos, una vez.
Fallido	totalRecaudado < meta	Cada aportante recupera exactamente lo suyo.

- 01

LEER EL CONTRATO

contracts/s08/Recaudo.sol: tres funciones, dos finales. Marquen cada if con un límite: son candidatos a borde.
- 02

NO LEER LOS MUTANTES

El ejercicio es pensar en los bordes. Abrirlos antes lo convierte en copiar, y no se aprende nada.

ESCRIBIR LA SUITE: DIEZ PRUEBAS O MÁS

GRUPO	MÍNIMO	QUÉ NO OLVIDAR
despliegue	2	Parámetros inválidos en el constructor.
aportar	3	Evento con argumentos exactos. Los límites del plazo.
recaudo exitoso	3	Permisos. Doble reclamo. Saldo exacto que recibe el dueño. Los límites de la meta.
recaudo fallido	3	Cada quien lo suyo. Quien no aportó. El dueño no se lleva nada. Repeticiones.

[] UNA PRUEBA FALLA POR LA RAZÓN CORRECTA, O NO PRUEBA NADA

Antes de dar por buena una prueba de reversión, pregúntense: ¿qué otra cosa podría hacerla revertir? Si una segunda llamada revierte porque el contrato se quedó sin dinero y no por la regla, la prueba pasa por accidente. Diseñen el escenario para que SOLO la regla pueda detenerla, y exijan el error exacto con `revertedWithCustomError`.



MEDIR LA COBERTURA Y LEER LO QUE FALTA

```
npx hardhat test test/s08/Recaudo.test.js --coverage
```

```
|| contracts\s08\Recaudo.sol | 11.54 | 15.79 | 49, 55-56, 60, 63-64, ... ← con la plantilla
```

01 SU FILA

contracts\s08\Recaudo.sol. Meta del curso: Line % $\geq$ 80.

02 CADA NÚMERO

Abran Recaudo.sol en esa línea. ¿Qué prueba la ejecutaría? Escribanla.

03 LO QUE QUEDA

Si solo quedan líneas de revert EnvioFallido, está bien: exigen un receptor que rechace el pago. Anótenlo en el informe.

Cómo saber que terminó esta parte: $\geq$ 10 pruebas en verde y Line % $\geq$ 80 en la fila de Recaudo.sol. Captura de pantalla.



CAZAR LOS TRES MUTANTES

```
node scripts/s08/cazar-mutantes.js
```

✓ Recaudo (correcto) 2 pruebas en verde

✗ VIVO RecaudoMutante1 2 pasan · 0 fallan

✗ VIVO RecaudoMutante2 2 pasan · 0 fallan

✗ VIVO RecaudoMutante3 2 pasan · 0 fallan

3 mutante(s) sobrevive(n). Hay un error que su suite no ve: piensen en los bordes.

01 MIENTRAS HAYA VIVOS

Volver a la tabla de bordes (A.17) y al subrayado de Recaudo.sol. Escribir la prueba que falta. No abrir el mutante.

02 CUANDO MUERAN LOS TRES

Ahora sí: abrir cada mutante, encontrar la diferencia y anotar qué prueba lo mató y por qué.



DESPLEGAR CON IGNITION EN LA RED LOCAL

SALIDA REAL, RECORTADA

```
npx hardhat ignition deploy ignition/modules/Recaudo.js
```

You are running Hardhat Ignition against an **in-process** instance **of** Hardhat Network.
This will execute the deployment, but the results will be lost.

Hardhat Ignition  

Deploying [RecaudoModulo]

Batch #1

Executed RecaudoModulo#Recaudo

[RecaudoModulo] successfully deployed  

Deployed Addresses

RecaudoModulo#Recaudo - 0x5FbDB2315678afecb367f032d93F642f64180aa3

Cómo saber que funcionó: «successfully deployed» y una dirección. En la red en memoria el resultado se pierde al terminar: es solo para comprobar que el módulo y los parámetros por defecto están bien, antes de gastar ETH de prueba.



KEYSTORE, DESPLIEGUE, VERIFICACIÓN Y USO

01 **SECRETOS** keystore set de SEPOLIA_RPC_URL (de Alchemy o Infura), SEPOLIA_PRIVATE_KEY (billetera del curso) y ETHERSCAN_API_KEY.

02 **DESPLEGAR** El comando de A.21: --network sepolia, --parameters y --verify. Pide la contraseña del keystore.

03 **COMPROBAR** La dirección en sepolia.etherscan.io muestra la marca de código verificado.

04 **USAR** Aportar 0,005 ETH desde Write Contract. Esperar los 30 minutos. Reclamar desde la cuenta que desplegó.



ANTES DEL GIT PUSH

git status: ni claves, ni .env, ni URLs con API key. Sí se sube ignition/deployments/chain-11155111: es la evidencia reproducible del despliegue.



LO QUE SUELE SALIR MAL, Y QUÉ SE ENTREGA

SÍNTOMA	CAUSA
El script dice «No se encontraron pruebas»	La suite no está en test/s08/Recaudo.test.js.
Una prueba de plazo pasa contra todo	Nunca llega al segundo exacto: revisen A.12.
Una reversión pasa por otra razón	Revisen el escenario (B.3) y exijan el error exacto.
loadFixture falla con una función anónima	La fixture necesita nombre: async function desplegar().
--verify falla	Falta ETHERSCAN_API_KEY, o reintentar el mismo comando tras un minuto.

EVIDENCIA	DETALLE	PESO
Suite	test/s08/Recaudo.test.js con ≥ 10 pruebas en verde.	25 %
Cobertura	Captura con ≥ 80 % de líneas en Recaudo.sol.	15 %
Mutantes	Los tres muertos + qué prueba mató a cada uno y por qué.	35 %
Despliegue	Dirección en Sepolia, verificada, desplegada con Ignition.	25 %



LO QUE EL LABORATORIO DEBIÓ DEJAR CLARO

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 Su suite tenía más de 80 % de cobertura antes de matar a los tres mutantes. ¿Qué les dice eso sobre el requisito del 80 %?

02 Para una de sus pruebas de borde: ¿qué otra razón podría haberla hecho fallar, y cómo lo descartaron?

03 ¿Qué líneas de Recaudosol quedaron sin cubrir, y por qué es aceptable (o no)?

04 Si mañana cambian un parámetro de Sepolia, ¿qué archivo tocan y qué comando repiten?



C



APROXIMADAMENTE 25 MINUTOS

EL PROYECTO, EN HARDHAT

Desde hoy el repositorio del equipo tiene la misma forma que el del curso. El Avance 1 se entrega en la Sesión 13.



LA ESTRUCTURA RECOMENDADA, DESDE HOY

```
npx hardhat --init # plantilla mocha + ethers
npm install --save-exact @openzeppelin/contracts@5.6.1

proyecto-equipo/
├── contracts/      su contrato principal (el borrador de la S7)
├── test/           una suite por contrato
├── ignition/modules/ un módulo por despliegue
├── ignition/parametros/ sepolia.json
├── docs/           anteproyecto, decisiones de diseño
├── hardhat.config.js con configVariable, sin secretos
├── package.json    versiones exactas
├── package-lock.json Sí se sube
├── .gitignore
└── README.md      qué hace, cómo instalar, cómo probar
```

- Usen las mismas versiones del curso (versiones.md): Hardhat 3.16.0, Solidity 0.8.34, OpenZeppelin 5.6.1.
- Repositorio en GitHub, los tres integrantes con permisos y el docente como colaborador.
- El borrador de la S7 se mueve a contracts/ y se le escriben pruebas con el método de hoy.



.GITIGNORE, Y LA REVISIÓN ANTES DE CADA PUSH

.GITIGNORE DEL CURSO

```
node_modules/  
artifacts/  
cache/  
coverage/  
types/  
ignition/deployments/chain-31337/  
.env  
*.log
```

NUNCA SE SUBE	POR QUÉ
Claves privadas y frases semilla	Vaciado en segundos. Y queda en el historial.
URLs de RPC con clave de API	Otro consume su cuota o la usa a su nombre.
node_modules, artifacts, cache	Se regeneran; pesan cientos de megas.
Datos personales (Ley 1581 de 2012)	Ni en la cadena ni en el repositorio: solo hashes.



LA RUTINA ANTES DE CADA GIT PUSH

git status y git diff --staged. Leer cada archivo que va a subir. Si aparece un secreto: no se sube, se rota la clave y se avisa. Borrarlo en un commit posterior NO lo elimina del historial.

EL ESTADO MÍNIMO DEL REPOSITORIO

PIEZA	ESTADO MÍNIMO ESPERADO PARA LA S9
Contrato principal	Compila en Hardhat. Aplica al menos dos patrones de la Sesión 7.
Suite inicial	Al menos 5 pruebas, con al menos una de cada familia de A.9 (incluido un borde).
Secretos	Ninguno en el repositorio. keystore configurado por quien despliega.
README	Qué hace el proyecto, cómo instalarlo y cómo correr las pruebas, en cinco líneas.

VERIFICACIÓN DEL BLOQUE C · CADA EQUIPO, ANTES DE SALIR

1. ¿La URL de su repositorio ya está en el canal del curso?
2. ¿Qué archivo de su repositorio podría filtrar un secreto si se descuidan?
3. ¿Qué borde de su contrato principal van a probar primero, y por qué ese?



LA PREGUNTA DE LA SEMANA

Si una suite con 88 % de cobertura dejó pasar tres errores, ¿qué número o qué evidencia le pedirían a un proveedor para creerle que su contrato está bien probado?

PARA LA PRÓXIMA SESIÓN

- Evidencia del laboratorio 08: suite, cobertura, mutantes muertos con su explicación, y Recaudo verificado en Sepolia.
- Trabajo autónomo: el proyecto en la estructura de C.1, con el contrato principal compilando y 5 pruebas.
- Traer el refactor de CertificadosUSB de la S7, ahora con su suite corriendo en Hardhat.
- Crear cuenta en ethernaut.openzeppelin.com y resolver el nivel 0: la Sesión 9 empieza en el 1.



LO QUE SE LLEVAN DE HOY

Una prueba verde no demuestra nada si también estaría verde con el error.

Hoy escribieron pruebas y las pusieron a prueba. La próxima sesión cambian de lado: van a atacar contratos ajenos para aprender a defender los propios.

SESIÓN 9 · SEGURIDAD DE CONTRATOS · ETHERNAUT · PARCIAL PRÁCTICO